Lab 2: Collaboration and quality

Tests, pull requests and code review

Mohammed Said Belaid

Duration: 1h30, in pairs

Part 0: Setup (10 min)

A: on GitHub, make your repository `devops-<nameA>-<nameB>` **public**: Settings > General > Danger Zone > Change repository visibility.

Both, each on your own laptop:

```
cd ~/devops/shop # B: ~/devops/devops-<nameA>-<nameB>
git switch main && git pull
python3 -m venv .venv # fails? sudo apt install python3-venv
source .venv/bin/activate
pip install pytest
```

Part 1: First tests (20 min, together on A's laptop)

Create `requirements-dev.txt` containing the line `pytest`.

Replace the content of `test_cart.py` with:

```
from cart import average, total, apply_discount

def test_average():
    assert average([10, 20]) == 15

def test_average_empty_cart():
    assert average([]) == 0

def test_total_empty_cart():
    assert total([]) == 0

def test_discount():
    assert apply_discount([100, 50], 10) == [90, 45]
```

```
pytest # 4 passed
git status # .pytest_cache/ appears
```

Add `.pytest_cache/` to `.gitignore`.

Now break the code on purpose: in `average` (`cart.py`), change `return 0` into `return None`.

```
python3 main.py # still fine...
pytest # ...but 1 failed: read the message
git restore cart.py
pytest # 4 passed
git add .gitignore requirements-dev.txt test_cart.py
```

```
git commit -m "Add first unit tests"
git push
```

B: `git pull`, then `pytest`.

Part 2: Protect `main` (10 min)

A, on GitHub: Settings > Rules > Rulesets > New ruleset > New branch ruleset.

- Ruleset name: `protect-main`; Enforcement status: **Active**
- Target branches: Add target > **Include default branch**
- Keep *Restrict deletions* and *Block force pushes*
- Check *Require a pull request before merging*, with **1** required approval
- Create

B: add a line `Lab 2 in progress` at the end of `README.md`, then:

```
git commit -am "Try to push directly to main"
git push # rejected: read the message
git reset --hard HEAD~1 # drop this commit
```

Part 3: Issue, pull request, review (25 min)

B is the client and the reviewer, **A** is the author.

B, on GitHub: Issues > New issue.

- Title: `Add shipping cost`
- Description: `Shipping costs 5.  It is free from 50 (before taxes).`

A:

```
git switch -c feature/shipping
```

Add at the end of `cart.py`:

```
FREE_SHIPPING_FROM = 50

def shipping(prices):
    """Shipping cost: free from 50, before taxes."""
    if sum(prices) > FREE_SHIPPING_FROM:
        return 0
    return 5
```

In `test_cart.py`, also import `shipping` in the first line, and add at the end:

```
def test_shipping_small_cart():
    assert shipping([20]) == 5

def test_shipping_big_cart():
    assert shipping([100]) == 0
```

```
pytest # 6 passed
git commit -am "Add shipping cost"
git push -u origin feature/shipping
```

On GitHub, open the pull request (Compare & pull request):

- Description: what it does, how to test it, and `Closes #<issue number>`
- Reviewers: **B**

B, review it:

- Read the issue, then the Files changed tab. Is the rule "free **from** 50" respected?
- Run the tests yourself:

```
git fetch
git switch feature/shipping
pytest
```

- Click `+` on the faulty line and write a comment. Then Review changes > **Request changes**.

A: answer the comment. Add a test `test_shipping_free_from_50` that checks `shipping([50]) == 0`, run `pytest` (red), fix `cart.py`, run `pytest` (green), then:

```
git commit -am "Make shipping free from exactly 50"
git push # the pull request is updated
```

B: check the new commit, then Review changes > **Approve**.

A: Squash and merge, then Delete branch. Check that the issue is closed.

Both:

```
git switch main && git pull
git branch -D feature/shipping
git lg
```

Part 4: A bug, test first (20 min)

Roles swap: **A** is the client and the reviewer, **B** is the author.

A, open an issue:

- Title: `Discount over 100% gives negative prices`
- Description: `apply_discount([100], 150) returns [-50.0]. A discount must be between 0 and 100.`

B:

```
git switch main && git pull
git switch -c fix/discount-limit
```

First reproduce the bug with a test. At the top of `test_cart.py`, add `import pytest`, and at the end:

```
def test_discount_over_100_is_refused():
    with pytest.raises(ValueError):
        apply_discount([100], 150)
```

```
pytest # 1 failed: the bug is reproduced
```

Then fix `apply_discount` in `cart.py`:

```
def apply_discount(prices, percent):
    if not 0 <= percent <= 100:
        raise ValueError("percent must be between 0 and 100")
    return [p * (1 - percent / 100) for p in prices]
```

```
pytest # all passed
git commit -am "Refuse discounts outside 0-100%"
git push -u origin fix/discount-limit
```

Open the pull request: `Closes #<issue number>`, reviewer **A**.

A: review it as in Part 3 (read, run `pytest` on the branch). Is a negative discount refused too? Approve or request changes.

B: Squash and merge, Delete branch. Both: `git switch main && git pull`.

Cheat sheet

Command / action	Purpose
`pytest`	Run all `test_*` functions of `test_*.py`
`pytest -v` / `pytest -k discount`	Verbose / only tests matching a name
`with pytest.raises(ValueError):`	Check that an error is raised
`git push -u origin <branch>`	Publish a new branch
`git fetch` then `git switch <branch>`	Get a colleague's branch to review it
`Closes #N` in a PR description	Close issue N when the PR is merged
Review changes > Approve / Request changes	Give your review decision